

★ WEEK 15 · PROBLEM SET

Finance Applications: Three Hands-On Problems

Weight the Past · Spread a Shock · Price a Tool Call

Each problem: Statement → Principles & Lab Guide → Hand Calculation → Claude Code Prompt

Small numbers you can compute with a pencil — then a metaprompt that lets Claude Code scale the same idea to real data. Hand first, machine second.

Every Problem Follows the Same Four Steps

Work top to bottom — do not skip the hand calculation, it is where the understanding lives

1 · Statement

the question

A concrete finance scenario with a tiny dataset you can see at a glance.

one slide

2 · Principles

the tools

The formulas involved and a short lab guide: what to compute, in what order.

one slide

3 · Hand Calc

pencil first

Work every number by hand in a table.
Small n , clean arithmetic, no calculator fear.

1–2 slides

4 · Claude Code

scale it up

A metaprompt: paste it into Claude Code and it builds the full lab on real data.

one slide

Hand first, machine second — you check the machine, never the other way around.

Where the Data Comes From

Public / in-script sources first — this week's demos are built inline; CSVs are hand-calc references

Problem	Primary source (public · no signup)	Provided fallback file
P1 · Attention	6-value series on the slides (in-script)	fml_w15_p1_series.csv (hand-calc)
P2 · Graph	4-node adjacency on the slides (in-script)	fml_w15_p2_graph.csv (hand-calc)
P3 · Agent EV	payoff table on the slides — pure pencil work	— (three strategies, no file needed)
Scaled labs	numpy-generated series and graphs, in-script	—

Course rule: if data is free online (yfinance, FRED, OpenML) or generated in-script, the prompt uses it.

P1

PROBLEM 1 · ATTENTION

Weight the Past by Hand

One series, three attention patterns — the TFT idea at pencil scale

Forecast Tomorrow From Six Days

past = (10.0, 10.5, 9.8, 12.0, 11.2, 13.5), day 6 most recent. Forecast with three weightings

Day	1	2	3	4	5	6
Past value	10.0	10.5	9.8	12.0	11.2	13.5
Recency raw score	1	2	3	4	5	6

Helper

- $\Sigma \text{past} = 67.0$ · $\Sigma \text{raw} = 21$.
- Weighted numerator $\Sigma(\text{raw} \cdot \text{past}) = 245.4$.

Your deliverables

- Flat forecast (equal weights) and recency forecast.
- The all-on-last-day forecast; a one-line reading of the ladder.

The Tool: a Forecast Is a Weighted Blend

Attention just chooses the weights — everything else is a dot product

$$\hat{y} = \sum_i a_i y_i \quad a_i = \frac{\text{raw}_i}{\sum_j \text{raw}_j} \quad \text{past} = (10.0, 10.5, 9.8, 12.0, 11.2, 13.5)$$

Reading it

- Weights are raw scores normalized to sum to 1.
- Flat weights = the plain average — attention switched off.
- A real TFT LEARNS the raw scores from data, per forecast.

Lab guide — per weighting

- Step 1: normalize the raw scores (divide by their sum).
- Step 2: multiply weight $\times$ value, day by day.
- Step 3: add up; compare the three forecasts.

Three Weightings, Three Forecasts

Same six numbers — the weights alone move the answer by 2.3

Weighting	Weights a	Arithmetic	Forecast
Flat	1/6 each	$67.0 / 6$	11.17
Recency	$(1,2,3,4,5,6) / 21$	$245.4 / 21$	11.69
All on last day	$(0, 0, 0, 0, 0, 1)$	1×13.5	13.50

flat: 11.17 recency: $\frac{245.4}{21} = 11.69$ all-on-last: 13.50

The Weights ARE the Model

From 11.17 to 13.50 without touching the data — attention is a dial over the past

11.17

Flat — every day
equal, the plain
average

11.69

Recency — the recent
rally pulls the forecast
up

13.50

All-on-last — attention
at its most extreme

learned

What a real TFT does
to the weight row —
from data

The TFT upgrade

- Weights per forecast, not fixed forever — and per input type.
- Known-future inputs (earnings dates) get their own channel.

The honest caveat

- Learned weights need data to learn from — W8's appetite applies.
- And the backtest of any forecaster answers to W10.

Scale It: Learn the Weights Instead of Assuming Them

Paste the metaprompt — Claude Code fits attention on data where the truth is planted

[INPUT] Prompt · Claude Code

```
Build a single Colab-ready Python script that:  
1) verifies the toy first: past = (10.0, 10.5, 9.8,  
   12.0, 11.2, 13.5) -> flat 11.17, recency 11.69,  
   last-only 13.50; (reference: fml_w15_p1_series.csv)  
2) generates 2,000 windows of length 6 where the truth  
   is  $y = 0.6 * \text{day6} + 0.4 * \text{day2} + \text{noise}$ ;  
3) fits softmax attention weights over the 6 lags by  
   gradient descent (numpy only) to predict y;  
4) prints and bar-plots the learned weights - they  
   should spike on lags 6 and 2;  
5) compares test MSE: flat average vs recency vs the  
   learned weights;  
6) three-sentence verdict: what 'attention learns  
   where to look' means, in this week's language.
```

[OUTPUT] What to expect

```
toy check: 11.17 / 11.69 /  
            13.50 exactly  
learned weights: two  
                 spikes, lags 6 and 2  
MSE: learned < recency <  
     flat
```

Your three-row ladder,
upgraded to weights the
data chose itself.

P2

PROBLEM 2 · GRAPH

Spread a Shock by Hand

Four nodes, three rounds — message-passing with a pencil

A Supplier Takes a Hit

Nodes A–D. Links: A–B, A–C, B–C, C–D. A takes a shock of 5; everyone else starts at 0. Run $h \leftarrow \frac{1}{2}h + \frac{1}{2} \cdot (Ah)/\text{deg}$ for three rounds

Node	Linked to	Degree	Start h
A (the supplier)	B, C	2	5.0
B	A, C	2	0
C (the hub)	A, B, D	3	0
D (the edge of the web)	C only	1	0

Your deliverables

- The full h vector after rounds 1, 2, and 3 (two decimals).
- The round in which D first moves, and one line on why exactly then.

The Tool: Blend Yourself With Your Neighbors' Average

Half memory, half neighborhood — repeated until the web has heard

$$h \leftarrow \frac{1}{2} h + \frac{1}{2} \frac{Ah}{\deg} \quad h_0 = (5, 0, 0, 0), \quad \deg = (2, 2, 3, 1)$$

Reading it

- $(Ah)_v$ adds up the neighbors' values; dividing by $\deg$ averages them.
- The $\frac{1}{2} / \frac{1}{2}$ blend keeps some memory of yourself each round.
- One round = news travels exactly one edge.

Lab guide — per round

- Step 1: for each node, average its neighbors' current h .
- Step 2: blend: half own value, half that average.
- Step 3: write the new vector; repeat three times.

Twelve Updates, One Traveling Shock

Watch the 5.0 dilute into the web — and reach D exactly on schedule

Round	A	B	C	D
Start	5.00	0	0	0
1	$\frac{1}{2} \cdot 5 + \frac{1}{2} \cdot 0 = 2.50$	$\frac{1}{2} \cdot 0 + \frac{1}{2} \cdot (5/2) = 1.25$	$\frac{1}{2} \cdot 0 + \frac{1}{2} \cdot (5/3) = 0.83$	$\frac{1}{2} \cdot 0 + \frac{1}{2} \cdot (0/1) = 0$
2	1.77	1.46	1.04	$\frac{1}{2} \cdot 0 + \frac{1}{2} \cdot 0.83 = 0.42$
3	1.51	1.43	1.13	0.73

node D: $0 \rightarrow 0 \rightarrow 0.42 \rightarrow 0.73$ (two hops away — the shock arrives in round 2)

Rounds = Radius of Hearing

D is two hops from A — and moved exactly in round 2. Structure set the schedule

2.50

A after round 1 — half the shock stays home

0.42

D's first nonzero — round 2, two hops, on schedule

2 hops

A→C→D: the only path the news could take

→ mean

Run forever and everyone converges — diversity fades

The finance reading

- Contagion timing follows the wiring: hubs hear early, edges late.
- A fraud ring is this picture run backward: values that co-move too well.

The GNN upgrade

- Real GNNs learn WHAT to pass and how to weigh each edge.
- Layers = rounds: a 2-layer GNN literally sees a 2-hop world.

Scale It: Contagion on a Bigger Web

Paste the metaprompt — Claude Code grows the graph and watches structure set the schedule

[INPUT] Prompt · Claude Code

```
Build a single Colab-ready Python script that:  
1) verifies the toy first: the 4-node graph, shock 5  
   at A -> h after rounds 1-3 matches (D: 0, 0.42,  
   0.73); (reference: fml_w15_p2_graph.csv)  
2) builds a 30-node random graph (each pair linked  
   with prob 0.12, seed 0); shocks one node with 5;  
3) runs 8 rounds; plots each node's value vs round,  
   colored by hop-distance from the shocked node;  
4) prints a table: hop distance vs the round of first  
   movement - confirm 'rounds = radius';  
5) adds one extra edge from the shocked node to the  
   farthest node and reruns - show the arrival time  
   collapse;  
6) three-sentence verdict: why 'structure is the  
   signal', in this week's language.
```

[OUTPUT] What to expect

```
toy check: D = 0 / 0.42 /  
           0.73 exactly  
curves: bands by hop  
       distance, moving in  
       order  
table: first-move round  
      = hop distance  
new edge: the far node  
         now moves in round 1  
  
Your 12 updates, run on  
a web worth drawing.
```

P3

PROBLEM 3 • AGENT

Price a Tool Call

Memory vs RAG vs search — expected value decides, stakes set the depth

Should the Agent Look It Up?

Task: state last quarter's EPS in a client note. Right = +10; wrong = -20; each tool call costs 1

Strategy	P(correct)	Tool calls
Answer from memory (the LLM alone)	0.60	0
RAG: retrieve and read the filing	0.95	1
Web search, then RAG the filing	0.99	2

Helper

- $EV = P \cdot 10 - (1-P) \cdot 20 - \text{cost}$.
- Then redo the table with $L = 2$ (a casual chat, not a client note).

Your deliverables

- EV of all three strategies at $L = 20$ and at $L = 2$.
- The winner at each stake, and the guardrail this justifies.

The Tool: Expected Value of a Tool Call

The agent's think-step is an EV comparison — stakes and hit-rates in, action out

$$EV = P \cdot V - (1 - P) \cdot L - \text{cost} \quad V = 10 \text{ (right)}, \quad L = 20 \text{ (wrong)}$$

Reading it

- $P \cdot V$ is the prize weighted by the odds of earning it.
- $(1-P) \cdot L$ is the hallucination tax — it scales with the stakes.
- Cost is small and certain; the tax is large and probabilistic.

Lab guide — per strategy

- Step 1: compute $P \cdot 10$ (the prize side).
- Step 2: subtract $(1-P) \cdot 20$ and the tool cost.
- Step 3: rank; repeat the whole table at $L = 2$.

Six EVs, Two Verdicts

High stakes: look it up. Low stakes: memory is suddenly fine

Strategy	EV at L = 20 (client note)	EV at L = 2 (casual chat)
Memory (P = 0.60, free)	$6 - 8 - 0 = -2.0$	$6 - 0.8 = +5.2$
RAG (P = 0.95, cost 1)	$9.5 - 1 - 1 = +7.5$	$9.5 - 0.1 - 1 = +8.4 \checkmark$
Search + RAG (P = 0.99, cost 2)	$9.9 - 0.2 - 2 = +7.7 \checkmark$	$9.9 - 0.02 - 2 = +7.88$

memory: -2.0 RAG: $+7.5$ search+RAG: $+7.7 \checkmark$ (look it up)

The Hallucination Tax Scales With the Stakes

Same brain, same tools — the cost of being wrong chose the strategy

-2.0

Memory at high stakes
— free, and still a
losing bet

+7.7

Search + RAG wins the
client note — barely
over RAG

+8.4

RAG wins the casual
chat — the 2nd call
stops paying

L

The one variable that
reordered everything

The guardrail it justifies

- High-stakes outputs REQUIRE the retrieval step — by policy, not mood.
- That is Week 12's NOT-FOUND valve, promoted to a standing rule.

The agent reading

- A good agent runs this table implicitly at every think-step.
- Tool budgets and stake tiers make the arithmetic explicit — and auditable.

Scale It: an Agent Policy Table Across All Stakes

Paste the metaprompt — Claude Code sweeps the stakes and draws the policy boundaries

[INPUT] Prompt · Claude Code

Build a single Colab-ready Python script that:

- 1) verifies the toy first: $(P, \text{cost}) = (0.60, 0)$, $(0.95, 1)$, $(0.99, 2)$, $V = 10 \rightarrow$ EVs at $L = 20$: $-2.0 / 7.5 / 7.7$ and at $L = 2$: $5.2 / 8.4 / 7.88$;
- 2) sweeps L from 0 to 100; plots the three EV lines and shades which strategy wins in each region;
- 3) prints the two crossover stakes (memory $\rightarrow$ RAG and RAG $\rightarrow$ search+RAG);
- 4) simulates 1,000 tasks at $L = 20$ under each fixed strategy (random correctness by P); reports the realized average payoff and worst single outcome;
- 5) adds a 'policy agent' that picks the EV-max strategy per task given its L (draw L uniform 0-100); show it beats every fixed strategy;
- 6) three-sentence verdict: why tooling depth should follow stakes, in this week's language.

[OUTPUT] What to expect

toy check: all six EVs
exact
policy map: memory zone
at tiny L , RAG in the
middle, search+RAG for
big L
crossovers printed
simulation: policy agent
on top, memory's worst
case ugliest

Your six EVs, promoted
to an agent policy.

Problem Set — Three Things Your Hands Now Know

You computed every number yourself; the code only repeats what you already understand

1

Attention ladder: 11.17 → 11.69 → 13.50

Same six numbers, three weightings — the weights ARE the model, and a TFT learns them from data.

2

The shock reached D in round 2 — exactly its hop distance

$h \leftarrow \frac{1}{2}h + \frac{1}{2} \cdot (Ah)/\text{deg}$: one edge of news per round. Contagion timing follows the wiring.

3

Tool EVs: -2.0 / +7.5 / +7.7 at stake 20 — look it up

The hallucination tax scales with stakes; at $L = 2$ memory turns positive. Tooling depth follows the cost of being wrong.

Hand calculation first — then let Claude Code scale the same idea to real data.